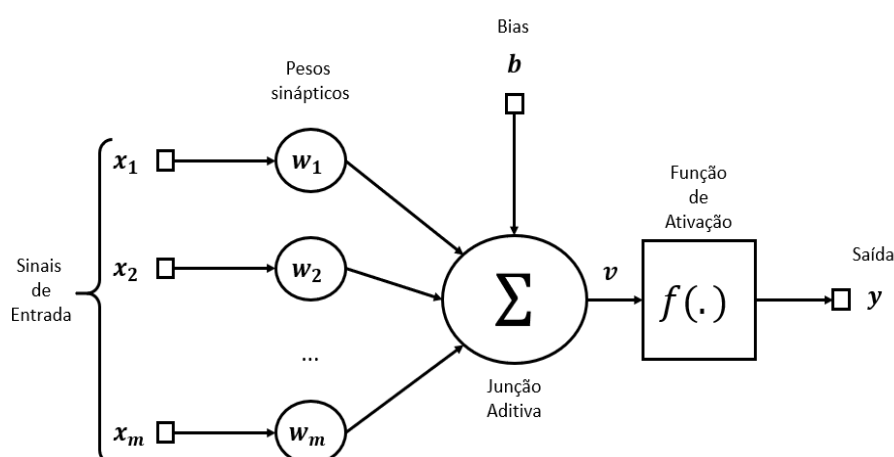


REDES NEURAIS ARTIFICIAIS E MACHINE LEARNING

TEMA 1 – O PERCEPTRON DE CAMADA ÚNICA

Compreender os conhecimentos relacionados com as RNA, tanto no aspecto de arquitetura quanto da dinâmica de aprendizagem e uso em situações concretas, requer o estudo da rede mais simples concebida, que é o **perceptron de camada única**. O perceptron se refere a um único neurônio artificial que pode conter várias entradas e uma única saída. As entradas são ponderadas por pesos sinápticos e somadas para produzir um sinal que será processado por uma função de ativação (função sinal). O perceptron de camada única utiliza o modelo de **neurônio artificial de McCulloch-Pitts** (Haykin, 2001, p. 161; Russell; Norvig, 2004).

Figura 1 – Perceptron de camada única



Fonte: Elaborado com base em Haykin, 2001, p. 162.

Um perceptron de camada única pode ser utilizado para classificação de um problema contendo duas classes, desde que as classes sejam **linearmente separáveis** (ou seja, se no espaço de entrada é possível separar as classes com um hiperplano; se houver duas entradas, o hiperplano é uma reta; se houver três entradas, é um plano em duas dimensões). Se atendida a condição do conjunto de dados ser linearmente separável, então o perceptron consegue classificar corretamente e sem erro o conjunto de entrada.

De acordo com a Figura 1, a ideia de um neurônio em um processo simples de classificação binária processa os m valores de entrada para produzir uma saída y . As duas classes no processo de classificação no perceptron são atribuídas ao sinal do valor de y , uma classe para valores positivos e outra para valores

negativos. Para obter essa classificação, os valores x_i devem ser ponderados pelos pesos sinápticos w_i . O resultado intermediário v é calculado, então, por:

$$v = \sum_{i=1}^m w_i x_i + b$$

Em que b é o peso sináptico fixo denominado **bias**, que, por sua vez, não pondera entrada nenhuma. O valor intermediário v é processado pela função de ativação $f(\cdot)$, que produz os valores $+1$ ou -1 para a saída y ,

$$y = f(v),$$

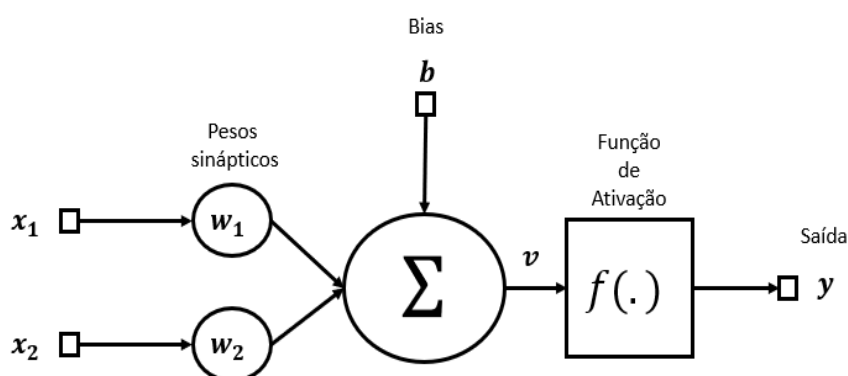
Em que:

$$f(v) = \begin{cases} +1, & v \geq 0 \\ -1, & v < 0 \end{cases}$$

Essa função de ativação é denominada **função sinal** (signal).

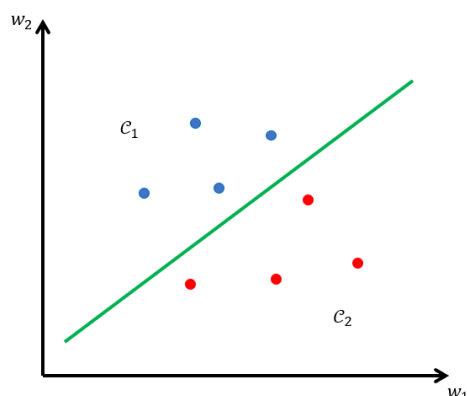
A representação de um perceptron de duas entradas é mostrada na Figura 2. Essa rede possui apenas dois pesos sinápticos, w_1 e w_2 . Com dois pesos, é possível fazer uma representação gráfica colocando cada peso sináptico como um eixo no plano cartesiano. Neste caso, com apenas duas entradas, o hiperplano é uma reta que divide o espaço das duas classes, $\mathcal{C}_1$ e $\mathcal{C}_2$ (Figura 3).

Figura 2 – Representação de um perceptron com duas entradas



Fonte: Medeiros, 2021.

Figura 3 – Representação do espaço de pesos do perceptron de camada única com duas entradas; a reta em verde é o hiperplano, que separa as duas classes



Fonte: Medeiros, 2021.

O somatório para as duas entradas pode ser simplificado para

$$v = w_1x_1 + w_2x_2 + b$$

Quando v é positivo ($v > 0$), as amostras de entrada pertencem à classe C_1 e estão acima da reta na figura 3. Pode-se afirmar, então, que:

$$w_1x_1 + w_2x_2 + b > 0$$

Quando v é negativo ($v < 0$), as amostras de entrada pertencem à classe C_2 e estão abaixo da reta na figura 3. Então:

$$w_1x_1 + w_2x_2 + b < 0$$

Quando $v = 0$, os valores coincidem exatamente com a reta¹ que divide as duas classes²:

$$w_1x_1 + w_2x_2 + b = 0 \quad (1)$$

Se isolarmos x_2 na equação (1):

$$x_2 = \frac{-w_1}{w_2}x_1 - \frac{b}{w_2} \quad (2)$$

¹ As máquinas de vetor de suporte buscam justamente calcular os parâmetros do hiperplano ótimo que divide as classes em um espaço de pesos.

² Convencionou-se também que o caso $v = 0$ deva pertencer a uma das classes. A separação feita aqui é puramente didática, no sentido de explicar a equação da reta.

Lembrando que a equação de uma reta (Fernandes, 2016) é expressa por (a variável x' aqui é a variável independente e y' a variável dependente, com m sendo o coeficiente angular e l o coeficiente linear):

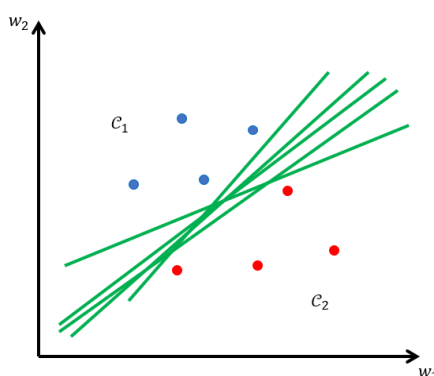
$$y' = mx' + l \quad (2.1)$$

Por analogia, pode-se verificar que a equação (2) isolada em x_2 é a equação da reta similar a (2.1) que divide o plano das amostras, desde que se faça $y' = x_2$, $x' = x_1$, o coeficiente angular seja $m = -w_1/w_2$ e o coeficiente linear $l = -b/w_2$.

O treinamento de um perceptron buscará, portanto, encontrar os melhores valores de w_1 e w_2 que permitem encontrar os parâmetros da reta que fará a divisão do plano em duas classes distintas. As amostras precisam ter previamente atribuídas as classes respectivas, e o treinamento se dá, então, de forma supervisionada. Quando novas amostras, que não fizeram parte deste treinamento, forem inseridas no perceptron, ele terá condições de inferir a qual classe estas novas amostras pertencerão.

Dessa forma, o perceptron com duas entradas permite uma visualização geométrica simples do processo de classificação. Quando se tem mais entradas e mais pesos, esta visualização fica dificultada, mas a noção da existência de um hiperplano separando as classes permanece válida. Outro fator importante a considerar é quanto ao uso de parâmetros diferentes para o perceptron: diferentes valores tenderão a gerar retas distintas para a separação das classes (Figura 4).

Figura 4 – Um feixe de infinitas retas pode separar as duas classes no espaço dos pesos sinápticos



Fonte: Medeiros, 2021.

TEMA 2 – O ALGORITMO DE TREINAMENTO LMS

Um dos algoritmos de treinamento que pode ser utilizado para encontrar os valores dos pesos sinápticos e tornar possível a classificação é o **algoritmo do mínimo quadrado médio** (*Least Mean Square*, ou LMS). O algoritmo LMS permite o aprendizado de forma gradativa do perceptron, atualizando os valores dos pesos sinápticos a cada época de treinamento e calculando um valor de erro global obtido a partir dos erros parciais de classificação do conjunto de amostras.

O algoritmo LMS foi criado por Widrow e Hoff ainda em 1960 (Widrow; Hoff, 1960). Sua implementação não é complexa e permite obter bons resultados no treinamento de um perceptron. Em suma, o algoritmo LMS consiste em calcular o erro de classificação das amostras de entrada e utilizar uma fração desse erro para ajustar dos pesos. Esse processo acontece até que se atinja um valor de erro global que seja o menor possível (Medeiros, 2018).

Em primeiro lugar, adota-se inicialmente valores aleatórios para os pesos sinápticos. Na primeira alimentação dos valores das amostras à entrada do perceptron, obtém-se o erro de classificação. De forma gradativa, um pequeno percentual desse erro é propagado de volta ao perceptron. Esse percentual é denominado **taxa de aprendizagem**. No que se refere aos erros, é necessário conhecer dois conceitos: o erro individual e o erro global.

O erro individual refere-se a cada classificação de amostra e tem a seguinte fórmula:

$$e_i = (o_i - y_i) \quad (3)$$

em que o erro e_i é a diferença entre a saída desejada o_i e a saída calculada após a ativação y_i . O erro global é uma medida de desempenho global do perceptron, sendo a média quadrática dos erros individuais:

$$E = \frac{1}{2} \sum_{i=1}^m e_i^2 \quad (4)$$

O aprendizado será gradativo e atualizará os pesos a cada execução da alimentação adiante. Para calcular a fração do erro que será realimentado no perceptron, utilizaremos a fórmula do delta:

$$\Delta_j = \eta e_i x_j \quad (5)$$

Em que:

- η = taxa de aprendizado;
- Δ_j = delta calculado;
- e_i = erro ou diferença;
- x_j = neurônio de entrada;
- n = número de amostras.

O índice i refere-se ao número de amostras, enquanto o índice j refere-se ao número de pesos sinápticos. Os deltas são calculados multiplicando-se o erro de cada amostra pela entrada referente ao delta. Logo após, é feita uma média dos valores calculados para os deltas. Após o cálculo, os pesos são atualizados com os valores dos deltas:

$$w_j(n+1) = w_j(n) + \Delta_j(n) \quad (6)$$

Por essa característica, tal regra de aprendizagem costuma ser também denominada **regra delta**.

Quando o algoritmo LMS converge ao longo das iterações (épocas de treinamento), ou seja, vai diminuindo o erro global a partir da diminuição gradativa dos erros locais, o valor do delta de atualização dos pesos vai se tornando menor. Uma diminuição gradativa dos erros indica que o perceptron está conseguindo alcançando o aprendizado, ou seja, está convergindo para valores dos pesos sinápticos que constituirão um hiperplano ótimo de classificação.

A taxa de aprendizagem é um parâmetro livre que desempenha um papel determinante para o treinamento do perceptron. É necessário encontrar um valor que atenda a uma relação de compromisso entre tempo de treinamento e qualidade da aprendizagem. Se a taxa de aprendizagem é muito pequena, o algoritmo LMS leva muito tempo para aprender. Se a taxa de aprendizagem é muito grande, pode haver perda de estabilidade e o algoritmo também pode não convergir adequadamente.

TEMA 3 – SIMULAÇÃO DO PERCEPTRON

Para ilustrar o treinamento de um perceptron de camada única e o adaline de maneira didática, utiliza-se um simulador escrito em HTML e Javascript³, que

³ Para o perceptron: <<http://professorfrontino.com.br/ia/simulator.html>>. Para o adaline: <<http://professorfrontino.com.br/ia/adaline.html>>.

simula um perceptron ou um adaline de duas entradas (dois pesos sinápticos). Adota-se um conjunto de seis amostras, com três amostras para cada classe e cria-se uma tabela para mostrar a alimentação adiante de cada neurônio, bem como o cálculo dos novos valores dos pesos sinápticos conforme o algoritmo LMS. Considera-se um problema de classificação em duas classes distintas, A e B.

3.1 Primeiro exemplo

Na Figura 5, encontra-se a tabela para o exemplo a ser trabalhado. Nos parâmetros consta a taxa de aprendizagem ($\eta = 0.01$) e os valores iniciais dos pesos w_1 e w_2 iguais a 1 (podem ser valores aleatórios), sendo o bias w_0 fixado em um valor específico (no caso, -1). O valor em *passo* dá a iteração atual do algoritmo e “limite” contém o número máximo de iterações que o algoritmo LMS pode executar.

Mais abaixo, cada linha representa uma amostra das 6 consideradas. Os valores de x_1 e x_2 são os valores das entradas, o valor d é calculado como a junção aditiva $d = w_1x_1 + w_2x_2 + w_0$. O valor de f é o resultado do valor de d após a função de ativação (+1 ou -1). O valor na coluna o significa qual a classe a que a amostra pertence (+1 se for da classe A ou -1 se for da classe B).

O erro e é calculado conforme a fórmula (3), sendo a diferença entre o valor da coluna o e o valor obtido na coluna f . Quanto existe erro de classificação, o campo fica em cor amarela, indicando qual o valor do erro; caso contrário, ficará com o valor zero, indicando que aquela amostra está sendo classificada corretamente. Portanto, nesse exemplo, pode-se identificar que três amostras estão classificadas corretamente com os valores atuais dos pesos sinápticos, enquanto três amostras estão classificadas incorretamente.

Os valores dos deltas, Δ_1 e Δ_2 são calculados de acordo com a equação (5) vista anteriormente. A taxa de aprendizagem é multiplicada pelo erro respectivo da amostra que é multiplicada, por sua vez, pela entrada respectiva. O valor da última linha (média dos deltas) é o que será adicionado aos pesos sinápticos w_1 e w_2 na parte superior da tabela para o próximo passo do algoritmo LMS. Ainda na última linha, consta também o valor do erro global de classificação, conforme a equação (4), que faz a soma dos erros elevados ao quadrado.

No simulador, podemos verificar o somatório dos erros quadráticos, resultando no erro global na última linha (no caso da Figura 5, $E = 12$). Entretanto, para ficar fiel à equação (4), esse valor ainda deveria ser dividido por 2. Sem perda

de generalidade, o valor mostrado no erro global será apenas o somatório dos erros quadráticos, não interferindo na análise global do algoritmo LMS⁴.

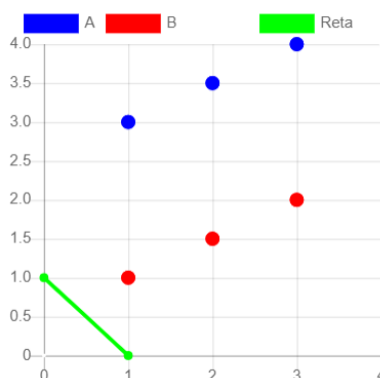
Figura 5 – Dados para o treinamento do perceptron de camada única

Simulador de Perceptron de Camada Única									
Parâmetros		η	0.01	Passo		0	Limite	100	
Pesos		w_1	1.000	w_2	1.000	w_0		-1	
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	3.00	1	1	0.00	0.00	0.00	0.00
2	1	1	1.00	1	-1	-2.00	4.00	-0.02	-0.02
3	2	1.5	2.50	1	-1	-2.00	4.00	-0.04	-0.03
4	2	3.5	4.50	1	1	0.00	0.00	0.00	0.00
5	3	4	6.00	1	1	0.00	0.00	0.00	0.00
6	3	2	4.00	1	-1	-2.00	4.00	-0.06	-0.04
Tempo(s)		0.25	Σ / Δ			12	-0.02	-0.01	

Fonte: Medeiros, 2021.

Pode-se notar, portanto, que as amostras que fazem parte da classe A são os pontos (1, 3), (2, 3.5) e (3, 4); na classe B estão os pontos (1, 1), (2, 1.5) e (3, 2). Quando projetados no plano cartesiano (Figura 6), os pontos são mostrados em cores diferentes e o hiperplano (reta em verde) é mostrado para os valores atuais dos pesos. Todos os pontos estão acima da reta, sendo demonstrado na Figura 5 como todos os valores da função de ativação, sendo +1.

Figura 6 – Representação no plano cartesiano tendo como dimensões os pesos sinápticos w_1 e w_2 ; o valor inicial 1 para os pesos sinápticos mostra uma reta que corta os eixos nos pontos (1, 0) e (0, 1)



Fonte: Medeiros, 2021.

⁴ A divisão do erro global por 2 permite que a expressão do delta seja da forma mostrada em (5). Entendendo-se o delta como uma aproximação da derivada parcial do erro quadrático em relação ao peso sináptico considerado, o valor 2 desapareceria da expressão final.

O algoritmo LMS será, então, executado de forma que os valores dos pesos sinápticos se modificarão gradativamente e a reta se adequará a uma posição na qual ela consiga dividir o plano entre as amostras representantes das classes A e B. A Figura 7 mostra o algoritmo no passo 32, onde os pesos sinápticos se adaptaram de maneira a classificar corretamente a amostra 2; o erro global caiu de 12 para 8.

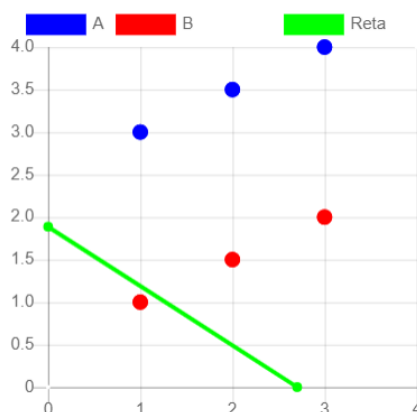
Figura 7 – Dados para o treinamento do perceptron de camada única no passo 32

Simulador de Perceptron de Camada Única									
Parâmetros		η	0.01	Passo		32	Limite	100	
Pesos		w_1	0.370	w_2	0.530	w_0		-1	
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	1.01	1	1	0.00	0.00	0.00	0.00
2	1	1	-0.07	-1	-1	0.00	0.00	0.00	0.00
3	2	1.5	0.59	1	-1	-2.00	4.00	-0.04	-0.03
4	2	3.5	1.67	1	1	0.00	0.00	0.00	0.00
5	3	4	2.33	1	1	0.00	0.00	0.00	0.00
6	3	2	1.24	1	-1	-2.00	4.00	-0.06	-0.04
Tempo(s)		1	Σ / Δ		8		-0.02	-0.01	

Fonte: Medeiros, 2021.

Na Figura 8, pode-se verificar que a reta agora está acima da amostra 2, dada pelo ponto (1, 1), classificando corretamente esta amostra.

Figura 8 – Representação no plano cartesiano dos pesos sinápticos no passo 32



Fonte: Medeiros, 2021.

A Figura 9 mostra o algoritmo LMS no passo 45, evidenciando que o perceptron já classifica corretamente a amostra 3; o erro global passou de 8 para

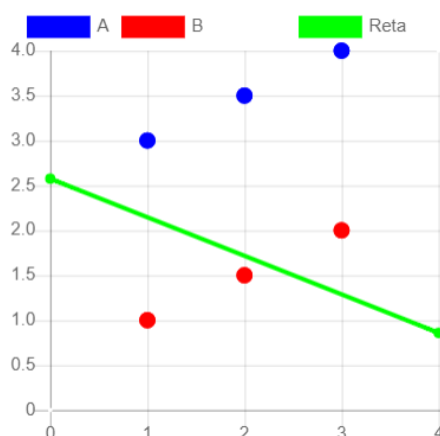
4, com os pesos sinápticos w_1 e w_2 tendo os valores 0.167 e 0.388, respectivamente. A Figura 10 mostra o plano com a reta acima das amostras 2 e 3.

Figura 9 – Dados para o treinamento do perceptron de camada única no passo 45

Simulador de Perceptron de Camada Única									
Parâmetros		η	0.01	Passo		45	Limite	100	
Pesos		w_1	0.167	w_2	0.388	w_0		-1	
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	0.36	1	1	0.00	0.00	0.00	0.00
2	1	1	-0.43	-1	-1	0.00	0.00	0.00	0.00
3	2	1.5	-0.05	-1	-1	0.00	0.00	0.00	0.00
4	2	3.5	0.74	1	1	0.00	0.00	0.00	0.00
5	3	4	1.11	1	1	0.00	0.00	0.00	0.00
6	3	2	0.32	1	-1	-2.00	4.00	-0.06	-0.04
Tempo(s)		1	Σ / Δ			4	-0.01	-0.01	

Fonte: Medeiros, 2021

Figura 10 – Representação no plano cartesiano dos pesos sinápticos no passo 45



Fonte: Medeiros, 2021.

Finalmente, no passo 53, o algoritmo convergiu, classificando corretamente todas as amostras (Figura 11). A amostra 6 é classificada corretamente, com o erro global sendo zero e os valores dos pesos sinápticos w_1 e w_2 tendo os valores 0.097 e 0.342, respectivamente. Nota-se que a reta está ligeiramente acima da amostra 6. Entretanto, ela é suficiente para dividir o plano e classificar corretamente as amostras em cada classe.

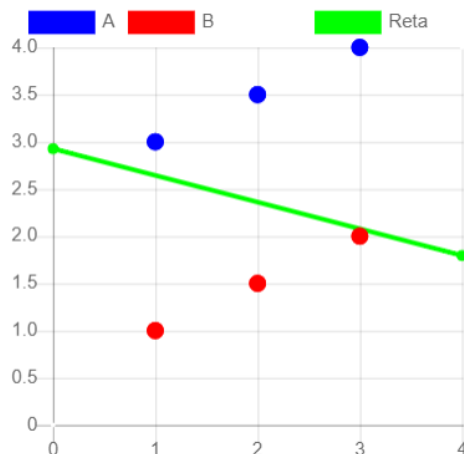
Figura 11 – Dados para o treinamento do perceptron de camada única no passo 45

Simulador de Perceptron de Camada Única									
Parâmetros			η	0.01	Passo		53	Limite	100
Pesos			w_1	0.097	w_2	0.342	w_0	-1	
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	0.12	1	1	0.00	0.00	0.00	0.00
2	1	1	-0.56	-1	-1	0.00	0.00	0.00	0.00
3	2	1.5	-0.29	-1	-1	0.00	0.00	0.00	0.00
4	2	3.5	0.39	1	1	0.00	0.00	0.00	0.00
5	3	4	0.66	1	1	0.00	0.00	0.00	0.00
6	3	2	-0.03	-1	-1	0.00	0.00	0.00	0.00
Tempo(s)			1	Σ / Δ			0	0.00	0.00

Fonte: Medeiros, 2021.

Isso não significa que os valores dos pesos sinápticos sempre serão esses. Caso os parâmetros iniciais sejam diferentes, pode-se obter uma reta diferente desta, que divida as duas classes corretamente.

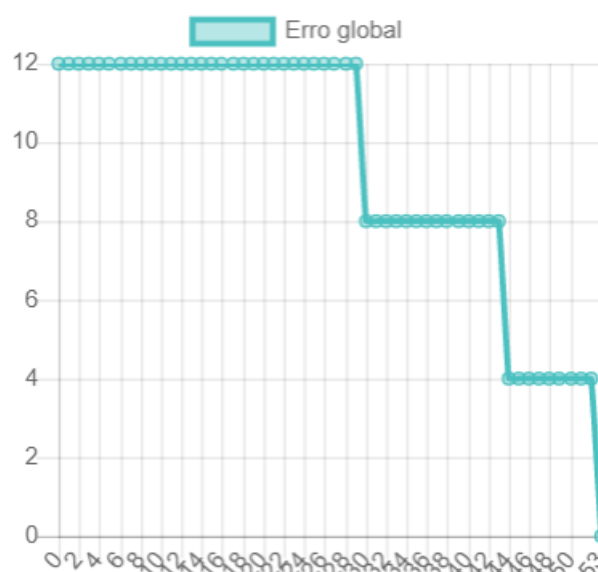
Figura 12 – Representação no plano cartesiano dos pesos sinápticos no passo 53



Fonte: Medeiros, 2021.

O gráfico do erro global pode ser visto na Figura 13. Pode-se verificar que o erro foi decrescendo de forma monótona até chegar a zero. No entanto, dependendo dos valores iniciais dos pesos e da taxa de aprendizagem, esse gráfico pode evoluir de maneira totalmente diferente.

Figura 13 – Gráfico da evolução do erro global durante o treinamento do perceptron



Fonte: Medeiros, 2021.

3.2 Exemplo com convergência rápida

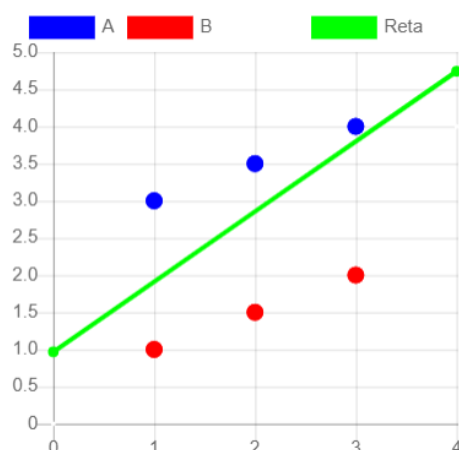
Neste exemplo, para a taxa de aprendizagem $\eta = 0.025$ e os valores iniciais para os pesos sinápticos w_1 e w_2 tendo os valores -1 e +1 respectivamente, o algoritmo LMS alcança o treinamento no passo 2, com os valores dos pesos - 0.975 e 1.033.

Figura 14 – Exemplo de execução do algoritmo LMS com parâmetros modificados, chegando-se à classificação correta com apenas dois passos

Simulador de Perceptron de Camada Única									
Parâmetros			η	0.025	Passo		2	Limite	100
Pesos			w_1	-0.975	w_2	1.033	w_0	-1	
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	1.13	1	1	0.00	0.00	0.00	0.00
2	1	1	-0.94	-1	-1	0.00	0.00	0.00	0.00
3	2	1.5	-1.40	-1	-1	0.00	0.00	0.00	0.00
4	2	3.5	0.67	1	1	0.00	0.00	0.00	0.00
5	3	4	0.21	1	1	0.00	0.00	0.00	0.00
6	3	2	-1.86	-1	-1	0.00	0.00	0.00	0.00
Tempo(s)			1	$\Sigma / \bar{\Delta}$			0	0.00	0.00

Fonte: Medeiros, 2021.

Figura 15 – Representação no plano cartesiano do exemplo que converge no passo 2



Fonte: Medeiros, 2021.

3.3 Exemplo com convergência lenta

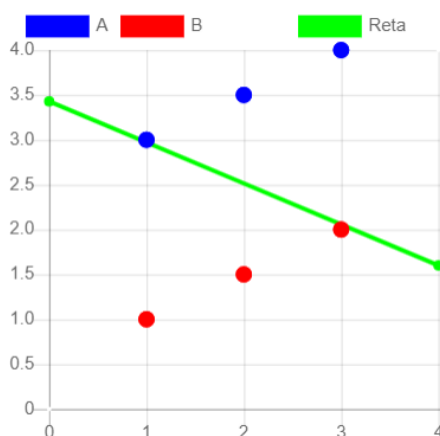
Neste exemplo, adota-se a taxa de aprendizagem $\eta = 0.025$ e os valores iniciais para os pesos sinápticos w_1 e w_2 tendo os valores +1 e -1 respectivamente, o algoritmo LMS alcança o treinamento no passo 65, com os valores dos pesos - 0.133 e 0.292 (Figura 16). Pode-se notar que a reta que separa as classes (Figura 17) é semelhante ao primeiro exemplo, mas o gráfico do erro global (Figura 18) mostra uma trajetória bem diferente do treinamento com o algoritmo LMS, com os valores do erro global assumindo valores discretos.

Figura 16 – Exemplo de execução do algoritmo LMS com convergência lenta, chegando-se à classificação correta com 65 passos

Simulador de Perceptron de Camada Única									
Parâmetros			η	0.025	Passo		65	Limite	100
Pesos			w_1	0.133	w_2	0.292	w_0		-1
No.	x_1	x_2	v	y	o	e	e^2	Δ_1	Δ_2
1	1	3	0.01	1	1	0.00	0.00	0.00	0.00
2	1	1	-0.57	-1	-1	0.00	0.00	0.00	0.00
3	2	1.5	-0.30	-1	-1	0.00	0.00	0.00	0.00
4	2	3.5	0.29	1	1	0.00	0.00	0.00	0.00
5	3	4	0.57	1	1	0.00	0.00	0.00	0.00
6	3	2	-0.02	-1	-1	0.00	0.00	0.00	0.00
Tempo(s)			0.1	Σ / Δ			0	0.00	0.00

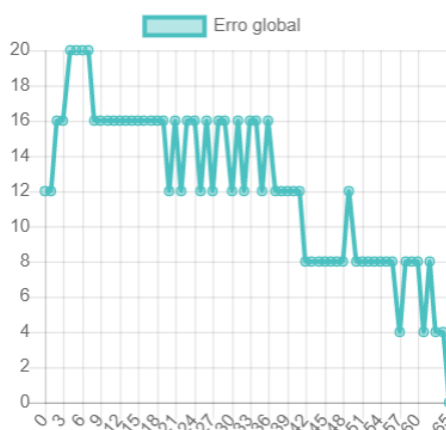
Fonte: Medeiros, 2021.

Figura 17 – Representação no plano cartesiano do exemplo com convergência lenta



Fonte: Medeiros, 2021.

Figura 18 – Gráfico da evolução do erro global durante o treinamento do perceptron com convergência lenta

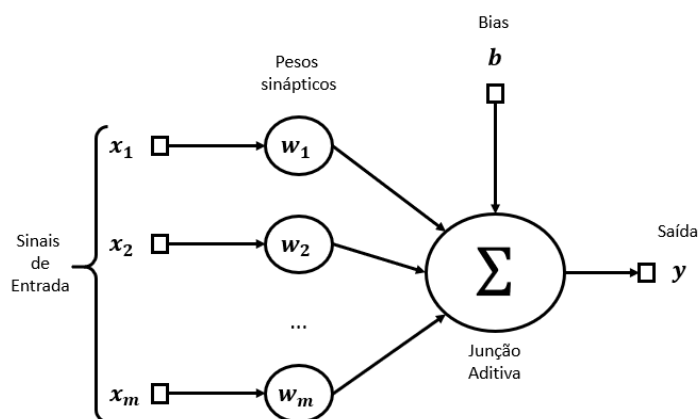


Fonte: Medeiros, 2021.

TEMA 4 – O ADALINE

No estudo do perceptron de camada única, o estudo de outro tipo de neurônio artificial, o **adaline** (*AD*Adaptive *LINE*ar element, que será denotado para fins de simplificação como *adaline*), auxilia na compreensão geral dos processos de treinamento das RNA. Na Figura 19 está representado esse tipo de neurônio. Pode-se identificar uma semelhança significativa com o perceptron, exceto pelo fato de que não há no adaline uma função de ativação: o resultado da junção aditiva é a própria saída do neurônio. Por isso, o adaline é considerado um neurônio artificial linear, enquanto o perceptron de camada única é não linear. O adaline foi criado também por Widrow e Hoff, e da mesma forma o algoritmo LMS pode ser utilizado para a minimização do erro entre as saídas e entradas da rede (Medeiros, 2018).

Figura 19 – Representação do adaline



Fonte: Medeiros, 2021.

A dinâmica do adaline é bastante semelhante ao processamento de sinais por um filtro linear. A adaptação dos pesos é feita por meio da retroalimentação de parte do sinal de saída, da mesma forma que o perceptron. No entanto, por não ter uma função de ativação que opera sobre o sinal da junção aditiva, o valor da saída y será um valor distinto para cada amostra, ao invés dos valores dicotômicos $+1$ e -1 resultantes do perceptron.

O menor erro global no caso do adaline conduz o treinamento para a formação de um hiperplano que tenderá a minimizar as distâncias de cada amostra até ele. Enquanto para o perceptron, basta operar sobre o sinal da junção aditiva e qualquer hiperplano que venha a separar as classes é então suficiente; para o adaline, há a busca de um hiperplano ótimo.

Pode-se mostrar isso para o exemplo tratado anteriormente no caso do perceptron, em que mais de uma reta permite dividir o espaço no limite entre as duas classes, A e B. Utilizando o adaline para o mesmo exemplo, a Figura 20 mostra que apenas uma reta é capaz de minimizar totalmente o erro global. Pode-se notar também que os pontos relativos às amostras ficam equidistantes da reta por uma unidade. No caso dos pontos da classe A, que estão acima da reta divisória, a distância é positiva; para os pontos da classe B, por sua vez abaixo da reta, a distância é considerada negativa.

Podemos notar que na Figura 20 tem-se três pontos sobre a reta que separa as classes. Conforme a geometria analítica, bastam dois pontos para compor uma equação da reta que passa por estes dois pontos. Esta reta será expressa de acordo com as equações (2) e (2.1) vistas anteriormente. Pegando-se os pontos $P(1,2)$ e $R(3,3)$, e substituindo-se os dois pontos em (2.1) desde

que $x' = x_1$ e $y' = x_2$. Tem-se, portanto, um sistema de equações da seguinte forma:

$$\begin{cases} 2 = m + l \\ 3 = 3m + l \end{cases}$$

Resolvendo as incógnitas m e l (coeficientes angular e linear, respectivamente), chega-se aos valores $m = 0.5$ e $l = 1.5$. na seguinte equação:

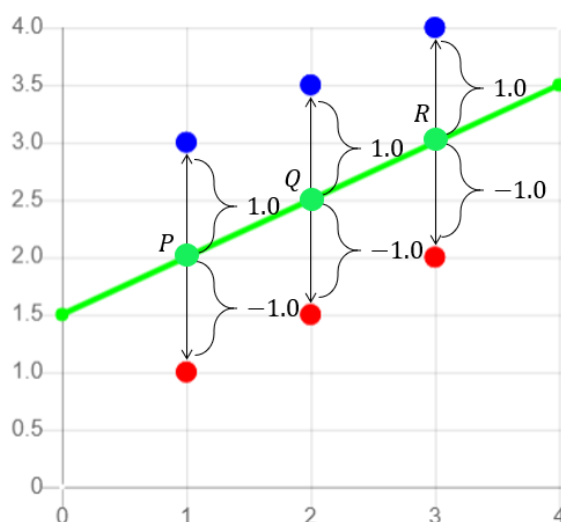
$$y' = 0.5x' + 1.5$$

Ou, substituindo as variáveis de maneira a parecer com a equação (1):

$$-0.5x_1 + x_2 - 1.5 = 0 \quad (7)$$

Portanto, desde que $w_1 = -0.5$ e $w_2 = 1$, com bias $w_0 = -1.5$, a reta será ótima no sentido de separar as duas classes, A e B.

Figura 20 – No treinamento do adaline, a reta em cor verde resulta no mínimo erro global, dividindo as classes em regiões equidistantes



Fonte: Medeiros, 2021.

Pode-se comprovar que esta reta é ótima, e o algoritmo LMS fará com que o erro global tenda a zero. Com exemplos mais complexos, em que as amostras estejam espalhadas pelas regiões, não se saberá os valores exatos dos pontos e o adaline buscará encontrar uma reta que divida da melhor forma as amostras representativas das classes.

TEMA 5 – SIMULAÇÃO DE UM ADALINE

A simulação de treinamento que foi executada anteriormente com o perceptron será feita também com o adaline. Os parâmetros do algoritmo LMS serão modificados com base nos coeficientes da reta divisória encontrada no tema anterior. A taxa de aprendizagem será $\eta = 0.025$, com os pesos começando de maneira que $w_1 = w_2 = 1$. Para o valor do bias, utilizar-se-á o próprio coeficiente linear que foi obtido no cálculo da reta do tema anterior, de forma que $w_0 = -1.5$. Pela figura 21, pode-se notar que os valores desejados o_i (coluna “o”) coincidem com as distâncias de cada amostra até a reta divisória, conforme a Figura 20. Note também que não há a coluna da função de ativação.

Figura 21 – Exemplo para o treinamento do adaline, no passo inicial

Simulador de Adaline								
Parâmetros			η	0.025	Passo	0	Limite	100
Pesos			w_1	1.000	w_2	1.000	w_0	-1.5
No.	x_1	x_2	y	o	e	e^2	Δ_1	Δ_2
1	1	3	2.50	1	-1.50	2.25	-0.04	-0.11
2	1	1	0.50	-1	-1.50	2.25	-0.04	-0.04
3	2	1.5	2.00	-1	-3.00	9.00	-0.15	-0.11
4	2	3.5	4.00	1	-3.00	9.00	-0.15	-0.26
5	3	4	5.50	1	-4.50	20.25	-0.34	-0.45
6	3	2	3.50	-1	-4.50	20.25	-0.34	-0.23
Tempo(s)			1	▼	$\Sigma / \bar{\Delta}$	63	-0.18	-0.20

Fonte: Medeiros, 2021.

Outro comentário a ser feito sobre a preparação para o treinamento do adaline é que os erros individuais podem assumir valores contínuos, ao invés dos valores mais discretos que eram obtidos no treinamento do perceptron. O adaline começa o algoritmo LMS com o erro global $E = 63$, muito maior que o erro global inicial do perceptron, que iniciou com $E = 12$.

Após iniciar a simulação, os dados do simulador no passo 20 apresentam-se conforme mostrados na Figura 22. Pode-se notar que os valores de y_i estão mais próximos dos valores de o_i , o erro global foi reduzido gradativamente para aproximadamente $E = 0.36$.

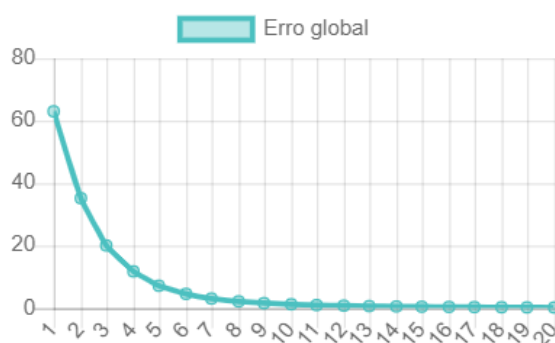
Figura 22 – Treinamento do adaline no passo 20

Simulador de Adaline								
Parâmetros			η	0.025	Passo	20	Limite	100
Pesos			w_1	-0.322	w_2	0.865	w_0	-1.5
No.	x_1	x_2	y	o	e	e^2	Δ_1	Δ_2
1	1	3	0.74	1	0.26	0.07	0.01	0.02
2	1	1	-0.96	-1	-0.04	0.00	-0.00	-0.00
3	2	1.5	-0.85	-1	-0.15	0.02	-0.01	-0.01
4	2	3.5	0.85	1	0.15	0.02	0.01	0.01
5	3	4	0.97	1	0.03	0.00	0.00	0.00
6	3	2	-0.73	-1	-0.27	0.07	-0.02	-0.01
Tempo(s)			1	Σ / Δ		0.189	-0.01	0.02

Fonte: Medeiros, 2021.

Pode-se verificar na figura 23, portanto, o comportamento mais contínuo da curva do erro global, mostrando a tendência para zero. À medida que o erro individual diminui, os valores dos deltas de atualização dos pesos também se reduzem, com o treinamento convergindo mais lentamente.

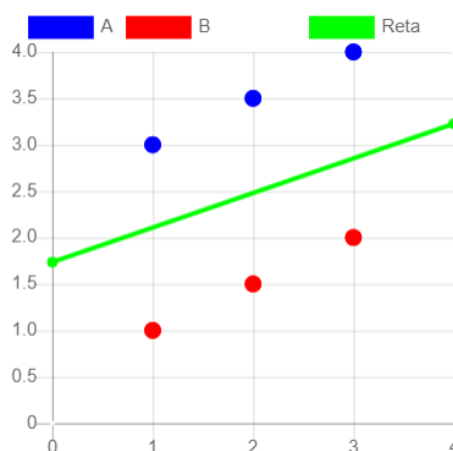
Figura 23 – Gráfico do erro global no treinamento com o adaline



Fonte: Medeiros, 2021.

A reta divisória no passo 20 pode ser vista na Figura 24, na qual se verifica estar muito próxima da reta ótima da figura 20. Isso quer dizer que, se fosse adotado o mesmo critério de parada para o perceptron, que tem base na **função sinal** (*signal*), o algoritmo LMS já teria treinado o adaline. No entanto, o erro global ainda não é mínimo, o que dá espaço para melhorar ainda mais os valores dos pesos.

Figura 24 – Representação do espaço de pesos sinápticos, mostrando a reta divisória no passo 20, já muito próxima da reta ótima da Figura 20



Fonte: Medeiros, 2021.

Por fim, na Figura 25, verifica-se que o adaline está treinado, com o erro global próximo de zero. Na prática, adota-se um valor mínimo para o critério de parada, o qual abaixo desta faixa o algoritmo LMS interrompe sua execução (nessa simulação, foi adotado o valor mínimo de 10^{-6} ; o erro global final foi $E = 9,6 \cdot 10^{-7}$). O erro global próximo de zero significa que as colunas y_i e o_i são coincidentes. Pode-se verificar também que os valores dos pesos, $w_1 = -0.5$ e $w_2 = 1.0$ coincidem com os coeficientes calculados para a reta conforme a equação (7).

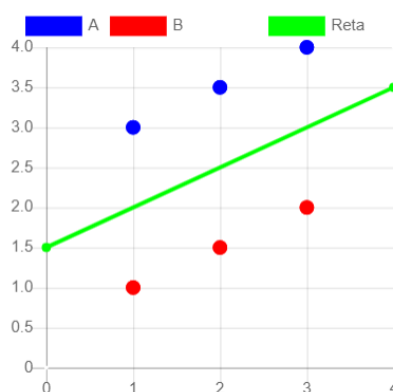
Figura 25 – Adaline treinado no passo 93

Simulador de Adaline								
Parâmetros			η	0.025	Passo	93	Limite	100
Pesos			w_1	-0.500	w_2	1.000	w_0	-1.5
No.	x_1	x_2	y	o	e	e^2	Δ_1	Δ_2
1	1	3	1.00	1	0.00	0.00	0.00	0.00
2	1	1	-1.00	-1	-0.00	0.00	-0.00	-0.00
3	2	1.5	-1.00	-1	-0.00	0.00	-0.00	-0.00
4	2	3.5	1.00	1	0.00	0.00	0.00	0.00
5	3	4	1.00	1	0.00	0.00	0.00	0.00
6	3	2	-1.00	-1	-0.00	0.00	-0.00	-0.00
Tempo(s)			1	Σ / Δ		9.608e-7	-0.00	0.00

Fonte: Medeiros, 2021.

O módulo da distância de cada amostra à reta divisória obtida era de uma unidade. Nesse exemplo, já se sabia o valor de o_i para cada amostra. Entretanto, no caso de um conjunto de amostras nas quais os pontos estejam distribuídos pelas regiões, fica difícil ou impraticável de se identificar quais valores seriam ótimos na avaliação dos pontos. Mesmo que se desconheça, é possível supor valores diferentes, desde que reflitam a diferença de sinal entre as classes: uma classe adota-se valores positivos de avaliação de y e para a outra classe valores negativos. Nesses casos, o algoritmo LMS irá aproximar da melhor forma uma reta que possa dividir as duas classes, ainda que o erro global não tenda a zero.

Figura 26 – Representação do espaço de pesos para o passo 93, mostrando a reta divisória ótima das classes A e B



Fonte: Medeiros, 2021.

Esse exemplo permitiu comparar a capacidade do perceptron e do adaline. Quanto ao problema da classificação binária, o perceptron parece ser mais robusto. Com a função de ativação, ele manifesta uma característica não linear, assumindo valores dicotômicos +1 e -1, e dessa forma, é mais robusto para tal tipo de problema, pois qualquer reta que divida um conjunto separável de amostras em duas classes irá servir para caracterizar a classificação. O adaline, por sua vez, demonstra comportamento linear e tenderá a fornecer aproximação.

Para conjuntos de amostras que não sejam separáveis, o perceptron não consegue convergir, havendo sempre um erro global residual. Em tais conjuntos, as classes não podem ter suas regiões divididas adequadamente por uma reta. Esse problema pode ser resolvido adicionando-se camadas ocultas ao perceptron, tornando-se assim um perceptron multicamada. O perceptron multicamada contém um número maior de pesos sinápticos, o que permite dividir uma região do espaço de pesos sinápticos com segmentos de retas.

REFERÊNCIAS

- FERNANDES, L. F. D. **Geometria Analítica**. Curitiba: InterSaberes, 2016.
- HAYKIN, S. **Redes Neurais**: princípios e prática. Porto Alegre: Bookman, 2001.
- MEDEIROS, L. F. de. **Inteligência Artificial Aplicada**: uma abordagem introdutória. Curitiba: InterSaberes, 2018.
- RUSSELL, S.; NORVIG, P. **Inteligência Artificial**. 2. ed. (tradução). Rio de Janeiro: Editora Campus, 2004.
- WIDROW, B.; HOFF, M. E. Adaptive Switching Circuits. *In*: , 1960. **Ire Wescon Convention Record**. [S. l.: s. n.], 1960. p. 96–104.